

Data Structures

Lecture 9: Basics of Tree

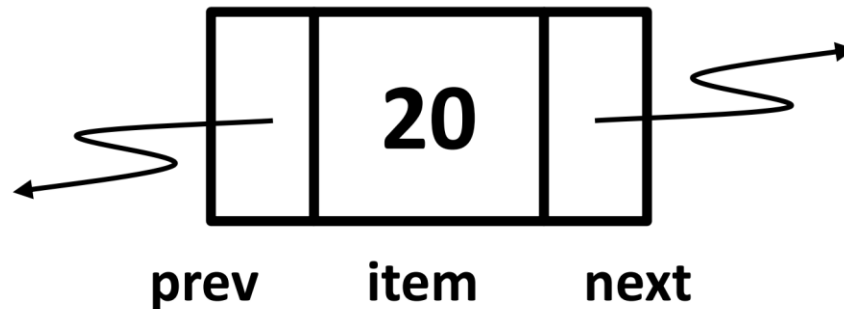
Soobin Um

Apr. 3, 2026

복습

Doubly Linked List

- 구조
 - 각 노드가 “이전 노드 (prev)”와 “다음 노드 (next)”에 대한 포인터를 모두 갖는 연결 리스트
- 특징
 - 양방향 탐색 가능
 - 삽입/삭제 연산 효율적
 - 메모리 사용량 증가



구현 예

```
public class DoublyLinkedList<E> {  
    /* DoublyLinkedList 클래스 내부에서만 접근 가능한 내부 클래스 */  
    private static class Node<E> {  
        E item;  
        Node<E> prev;    // 이전 노드 포인터  
        Node<E> next;    // 다음 노드 포인터  
  
        Node(E item) {    // 생성 시 item만 받음  
            this.item = item;  
            this.prev = null;  
            this.next = null  
        }  
    }  
    private Node<E> head;  
    private Node<E> tail;  
    private int size;  
  
    public DoublyLinkedList() {  
        head = null;  
        tail = null;  
        size = 0;  
    }  
    // Operations  
}
```

삽입: 구현

```
public void insert(int pos, E item) {  
    Node<E> newNode = new Node<>(item);  
    Node<E> curr = head;  
  
    for (int i=0; i<pos; i++)  
        curr = curr.next;  
  
    Node<E> prevNode = curr.prev;  
  
    newNode.next = curr;  
    newNode.prev = prevNode;  
    prevNode.next = newNode;  
    curr.prev = newNode;  
  
    size++;  
}
```

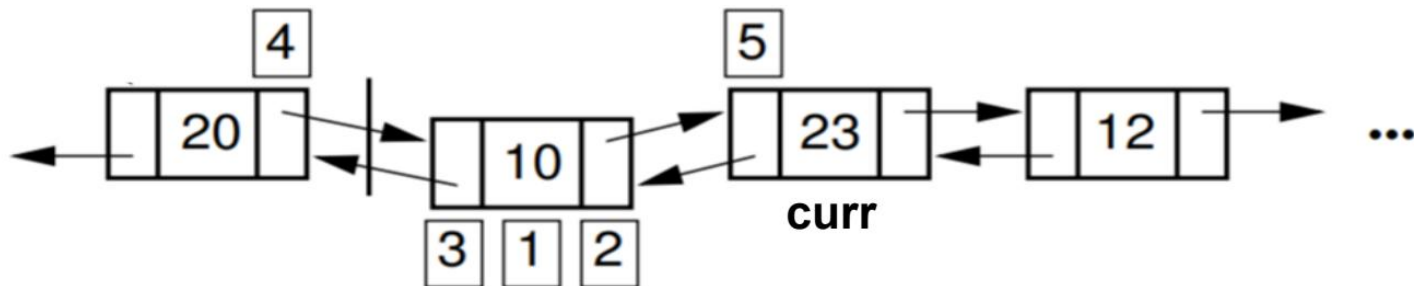
1

2

3

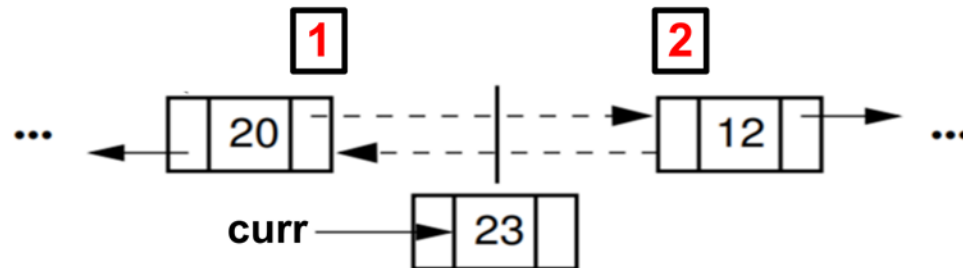
4

5



삭제: 구현

```
public E delete(int pos) {  
    Node<E> curr = head;  
  
    for (int i=0; i<pos; i++)  
        curr = curr.next;  
  
    Node<E> prevNode = curr.prev;  
    Node<E> nextNode = curr.next;  
  
    prevNode.next = nextNode; 1  
    nextNode.prev = prevNode; 2  
  
    size--;  
    return curr.item;  
}
```



Dictionary: 정의 및 구조

- 정의

- 데이터를 **Key-Value 쌍으로 저장**하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음 (즉, “**key 기반의 리스트**”와 같음)

- 구조

- 내부적으로는 배열 구조. (**배열의**) 각 칸에 **key-value 쌍을 함께 저장**
- Value는 중복 가능하지만, **key는 유일해야 함**
- Key를 **해시함수(Hash function)**를 통해 (**배열의**) 인덱스로 변환
 - Key 자체를 배열의 직접적인 인덱스로 사용할 수는 없기 때문

Key → [Hash Function] → Index

Index	0	1	2	3	4	5	6	7
Slot	null	(2025001, "Alice")	(2025017, "Cindy")	null	(2023049, "Jack")	null	null	(2022057, "John")

주요 연산 및 특징

- 주요 연산

- 삽입 (put): 새로운 key-value 쌍 추가
- 탐색 (get): key를 통해 value를 찾음
- 삭제 (remove): 특정 key-value 쌍 제거
- 수정 (put): 동일 key에 새로운 value 할당
- 키 존재 여부 확인 (contains): 딕셔너리 상에서 특정 key의 존재 여부 확인

- 특징

- 빠른 탐색

- 일반적인 리스트에서 특정 값을 찾을 땐, 기본적으로 순차 탐색 필요 ($O(n)$)
 - 하지만 딕셔너리에선 (key를 이용하므로) 평균 $O(1)$ 에 탐색 가능

- 순서 없음

- 일반적인 리스트는 구조적으로 앞에서 뒤 또는 뒤에서 앞으로의 순서가 존재
 - 하지만 딕셔너리는 순서를 유지하지 않음 ($\rightarrow$ 정렬을 지원하지 않음)

- 해시 충돌 (hash collision)

- 서로 다른 두 개 이상의 key가 해시 함수를 통해 같은 인덱스로 변환되는 상황
 - 삽입/삭제/탐색 연산 모두 최대 $O(n)$ 의 비용이 발생할 수 있음

구현: HashMap 구조 이용

```
import java.util.HashMap;

public static void main(String[] args) {
    HashMap<String, String> dict = new HashMap<>();

    dict.put("2025001", "Alice");    // 삽입
    dict.put("2025017", "Cindy");
    dict.put("2023049", "Jack");
    dict.put("2022057", "John");

    System.out.println("2025017: " + dict.get("2025017"));    // 탐색

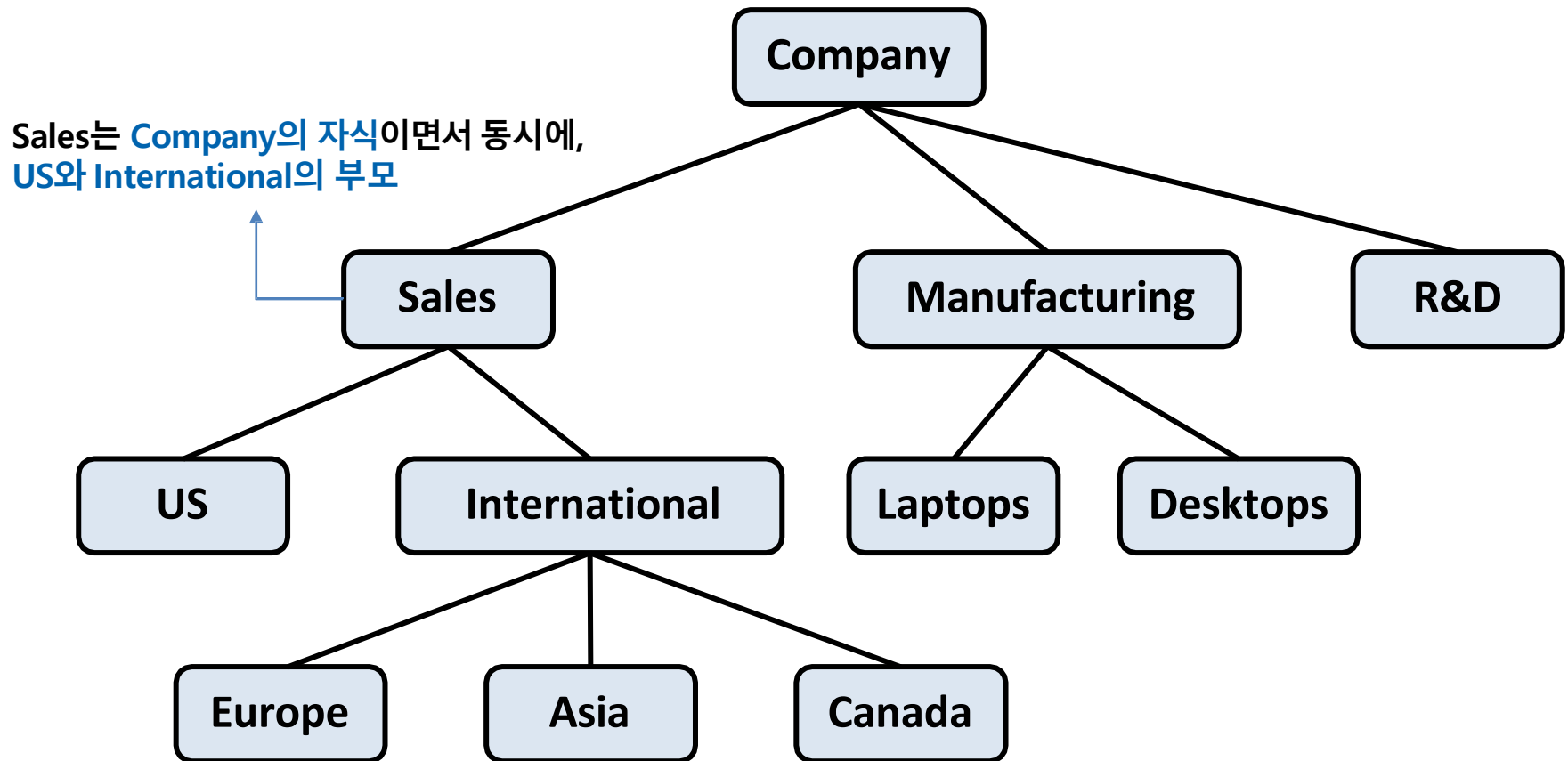
    dict.put("2025001", "Bob");    // 수정
    dict.remove("2023049");    // 삭제

    if (dict.containsKey("2025005"))    // 키 존재 여부 확인
        System.out.println("Exists")
}
```

Tree

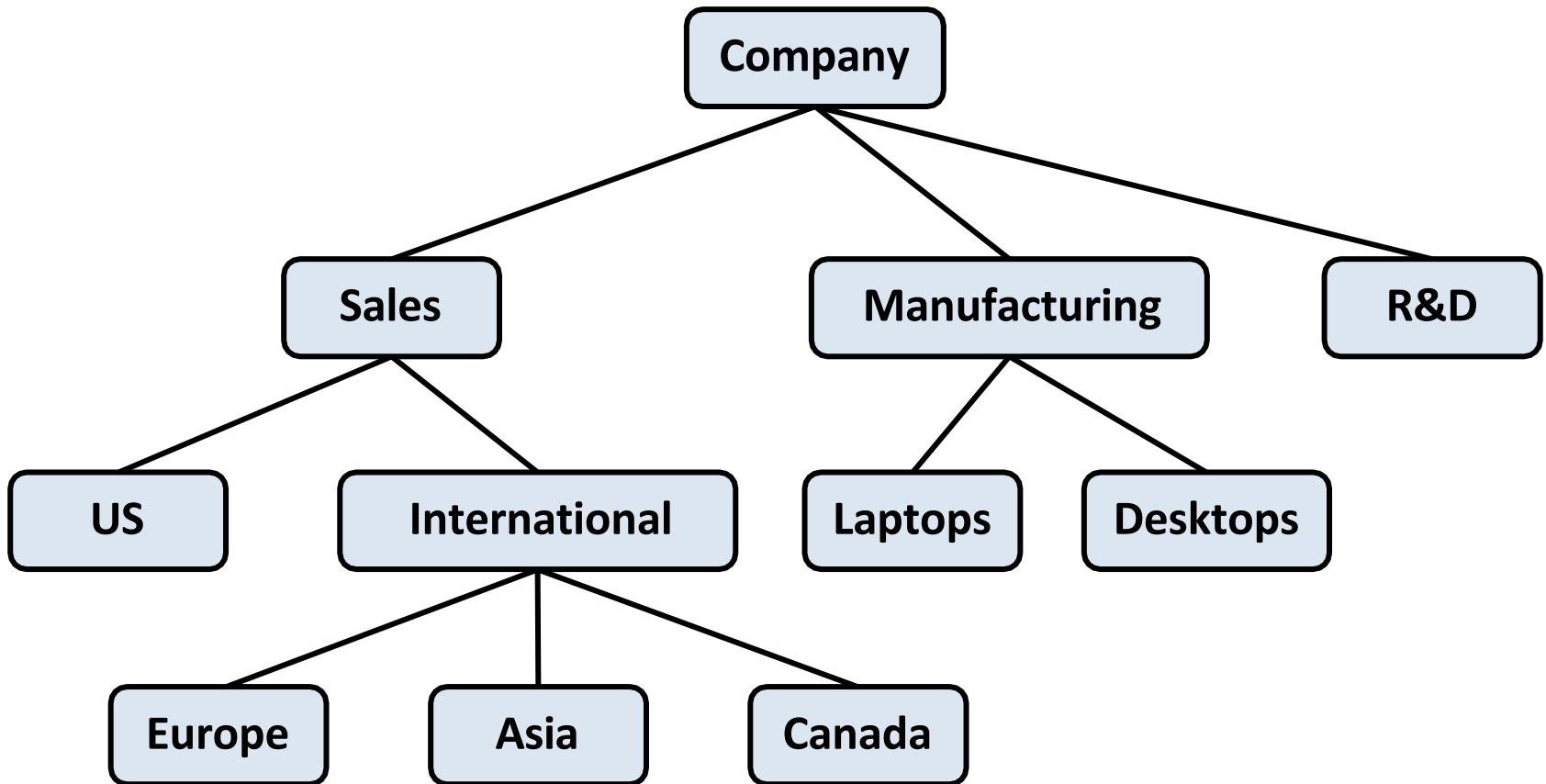
Definition of a Tree

- 데이터를 “계층적 (hierarchical)” 방식으로 저장하는 구조
 - 부모-자식 (Parent-Child) 관계를 통해 연결된 노드들로 구성됨 (예: 조직도, 파일시스템)



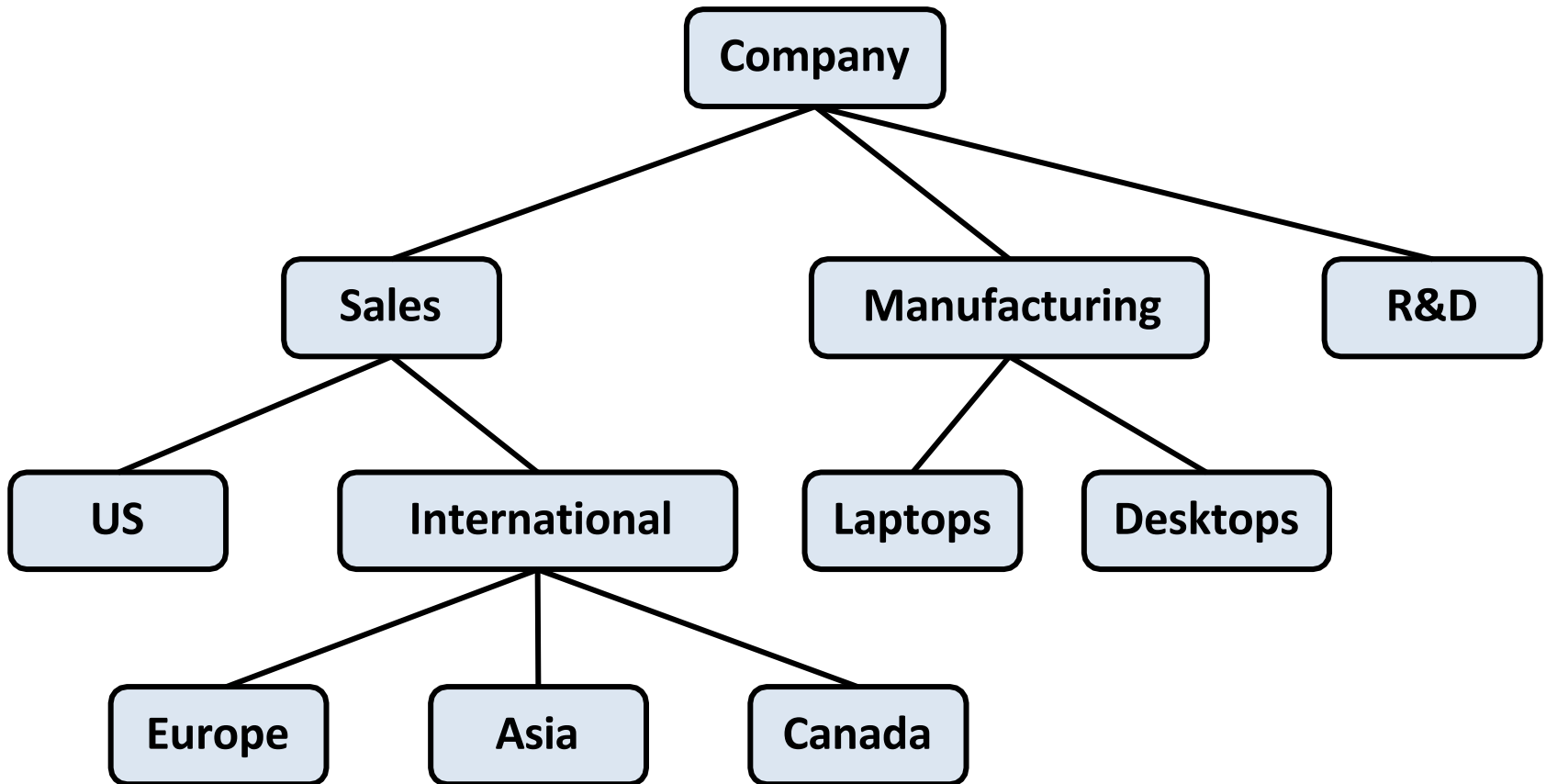
Terminologies

- **노드 (Node):** 데이터가 저장되는 기본 단위 (unit)
 - 노드에는 데이터 값과 다른 노드와의 연결 정보(포인터)가 포함될 수 있음
- **간선 (Edge):** 두 노드 사이의 연결 (relationship)



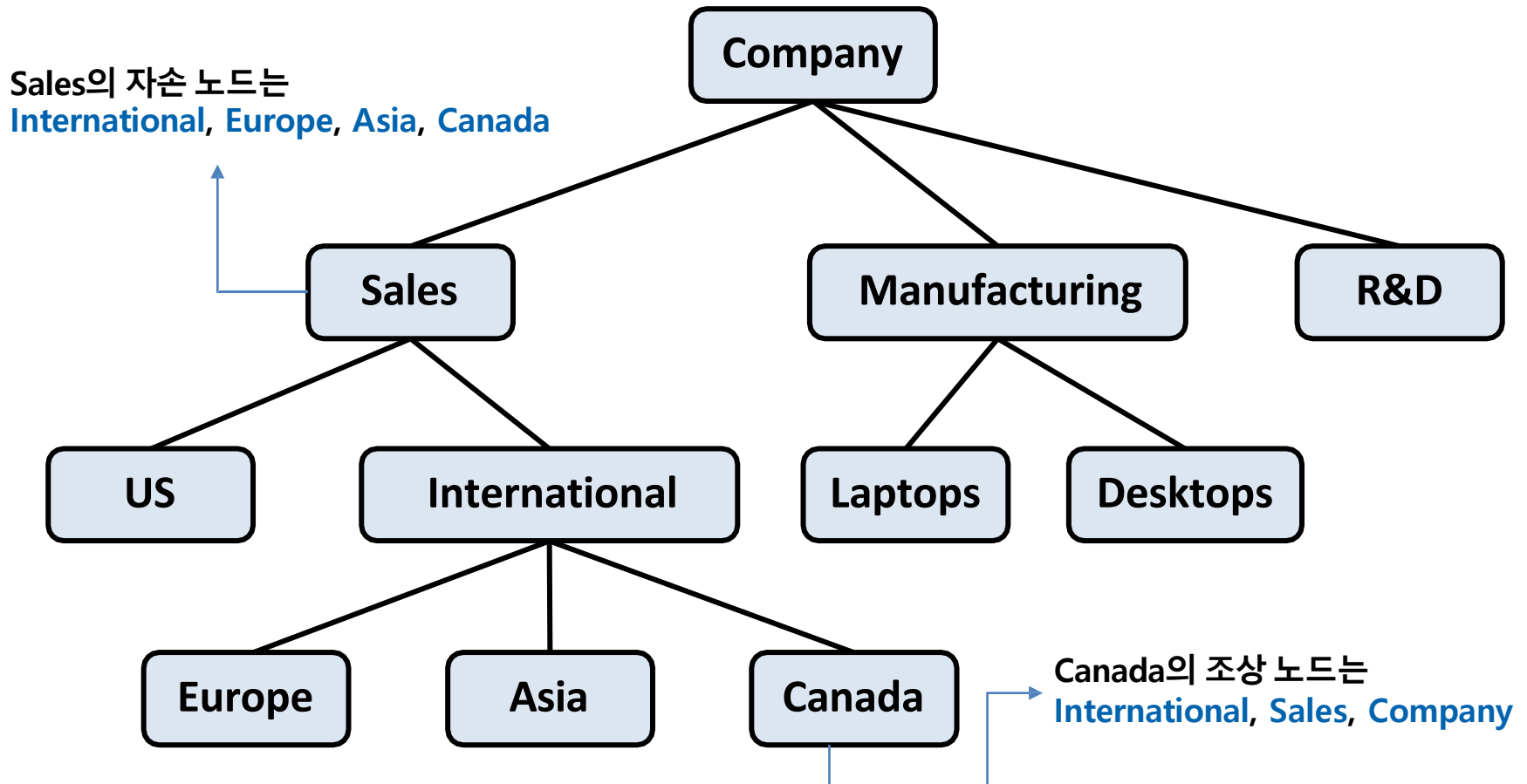
Terminologies

- 루트 (Root): 가장 상위에 있는 (부모 노드가 없는) 노드
- 내부 노드 (Internal node): 하나 이상의 자식 노드를 갖는 노드
- 외부 노드 (External node, leaf node): 자식 노드가 없는 노드



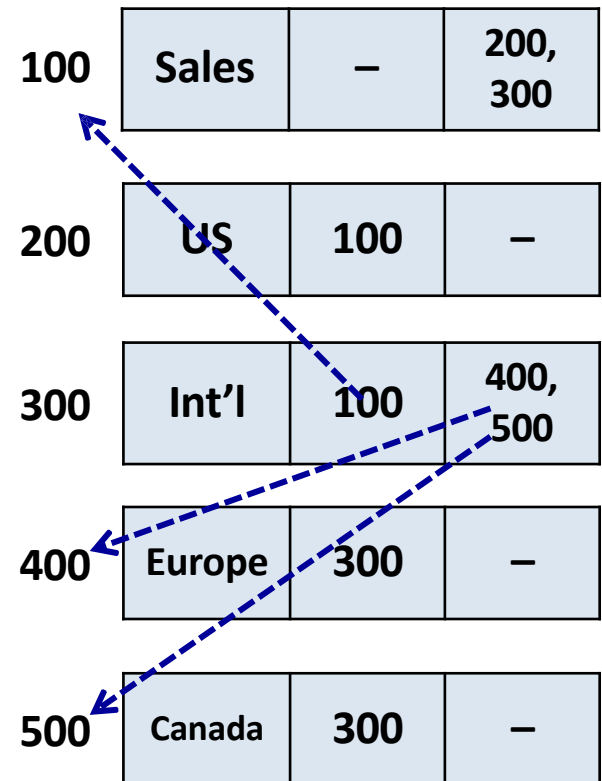
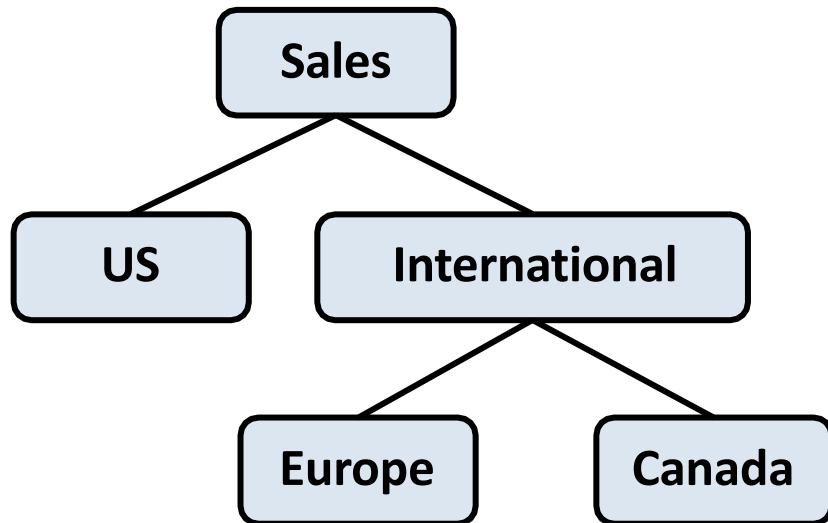
Terminologies

- **조상 노드 (Ancestor)**
 - 어떤 노드에서 루트까지 가는 경로 상에 존재하는 모든 노드 (부모 노드는 조상 노드에 포함)
- **자손 노드 (Descendant)**
 - 어떤 노드로부터 하위 방향으로 이어지는 모든 노드



Tree in Memory (Linked Structure)

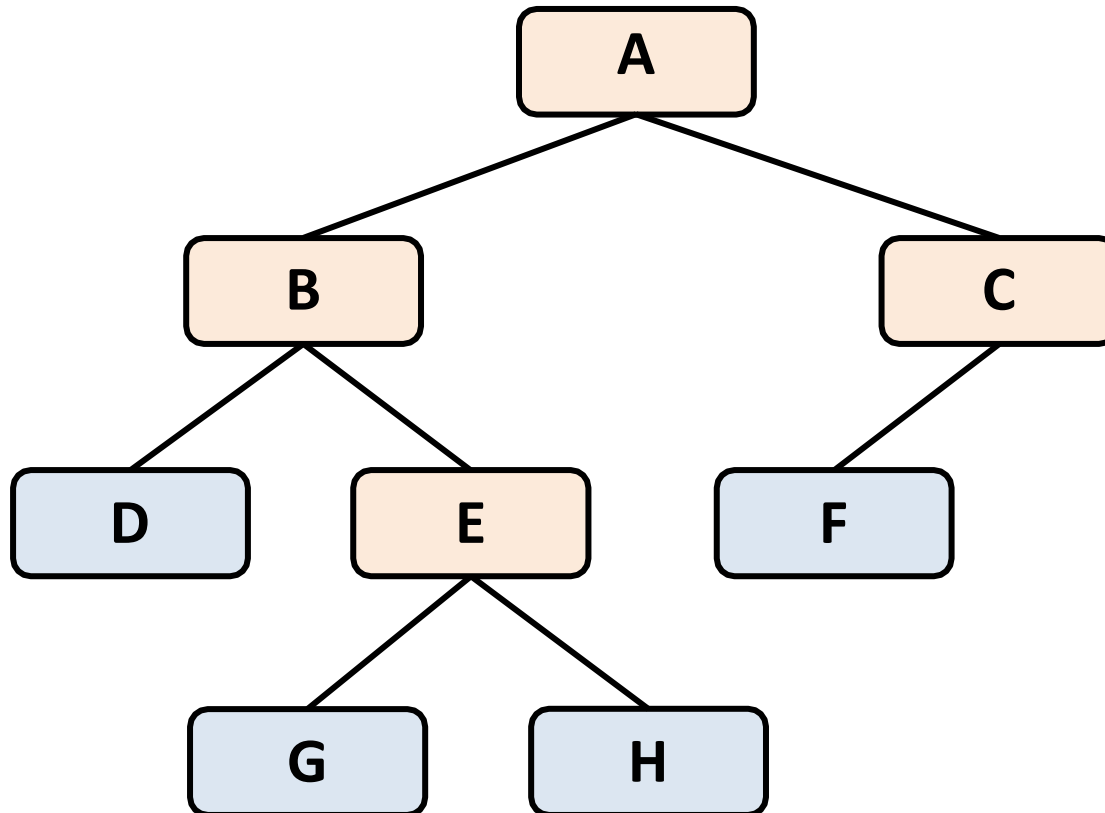
- 노드는 다음을 저장하는 객체(object)로 표현:
 - 데이터 원소 (data element)
 - 부모 노드에 대한 연결 (link to parent)
 - 자식 노드들의 집합 (set of children)



Binary Tree

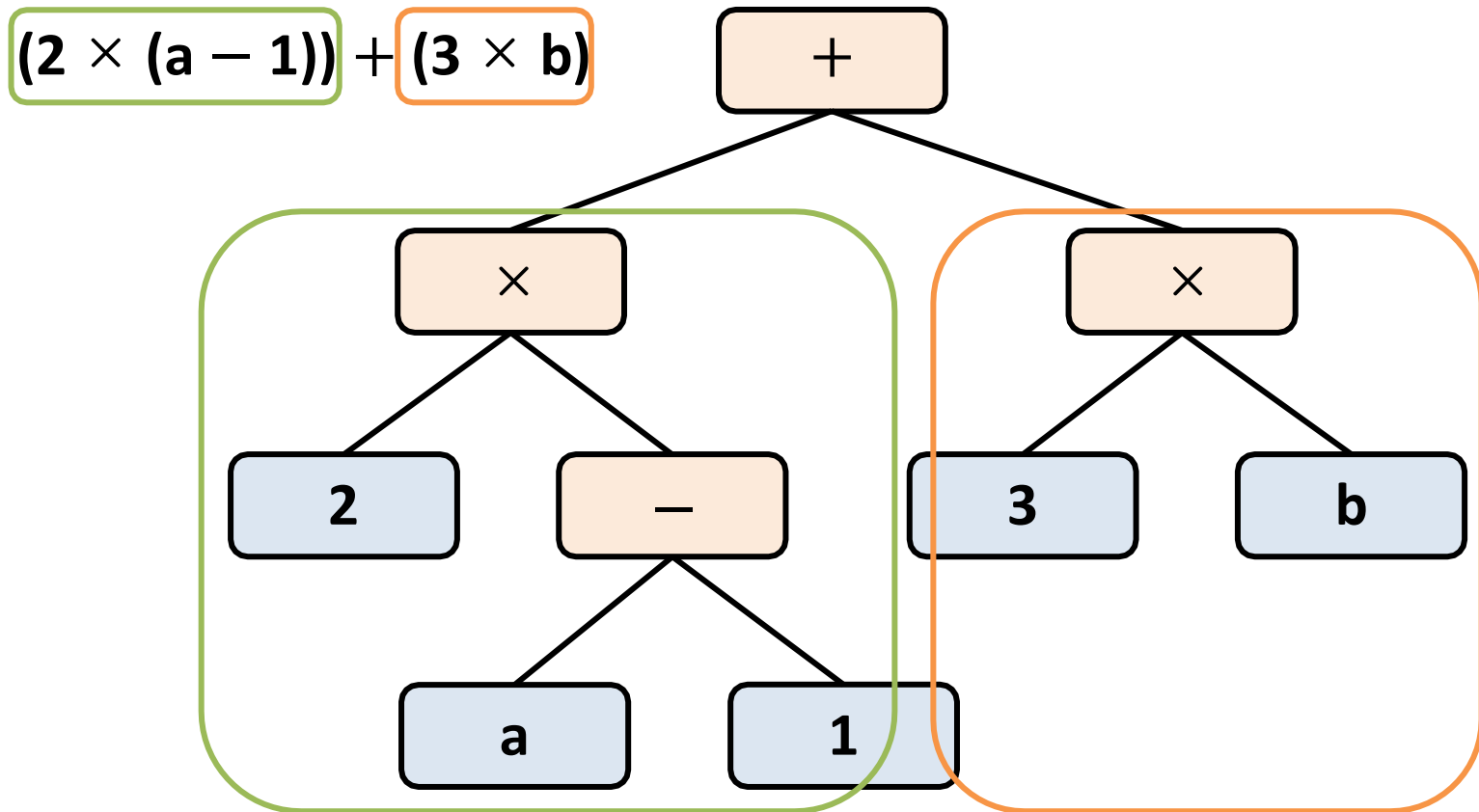
Binary Tree (이진 트리)

- 각 내부 노드는 최대 두 개의 자식을 가짐
- 한 노드의 자식들은 순서가 정해져 있음



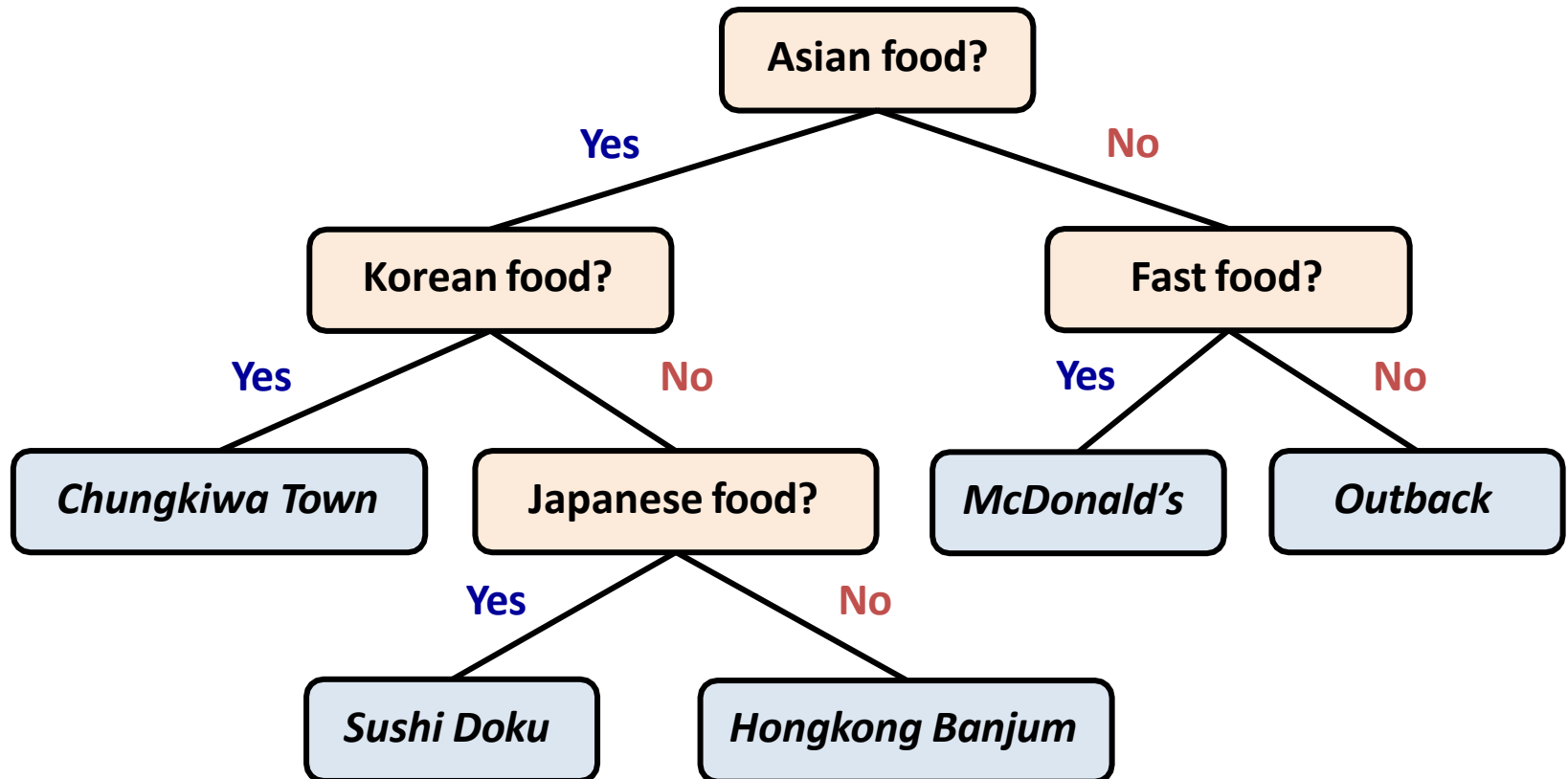
Ex 1) Arithmetic Expression Tree (산술 표현식 트리)

- 내부 노드: 연산자 (operator)
- 외부 노드: 피연산자 (operand)



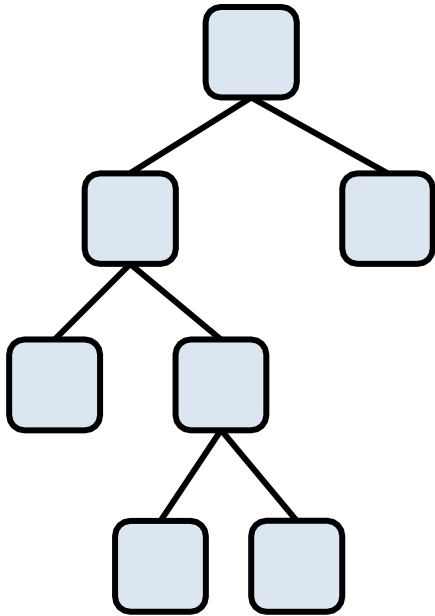
Ex 2) Binary Decision Tree (이진 결정 트리)

- 내부 노드: Yes/No로 대답할 수 있는 질문 (또는 조건)
- 외부 노드: 최종 결정 (또는 클래스 레이블이나 예측 값)

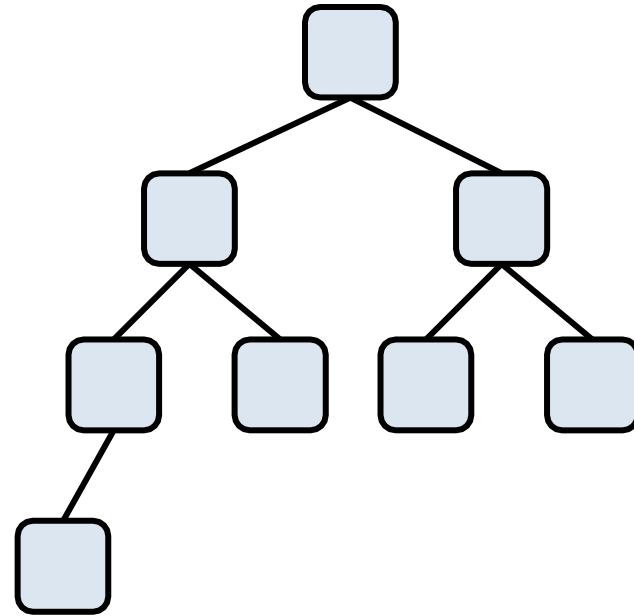


Types of Binary Trees

- 정 이진 트리 (Full Binary Tree)
 - 각 노드의 자식 노드 수가 2 또는 0인 트리
- 완전 이진 트리 (Complete Binary Tree)
 - 가장 깊은 레벨을 제외한 모든 레벨이 가득 차 있음
 - 마지막 레벨의 노드들은 가능한 왼쪽에 존재



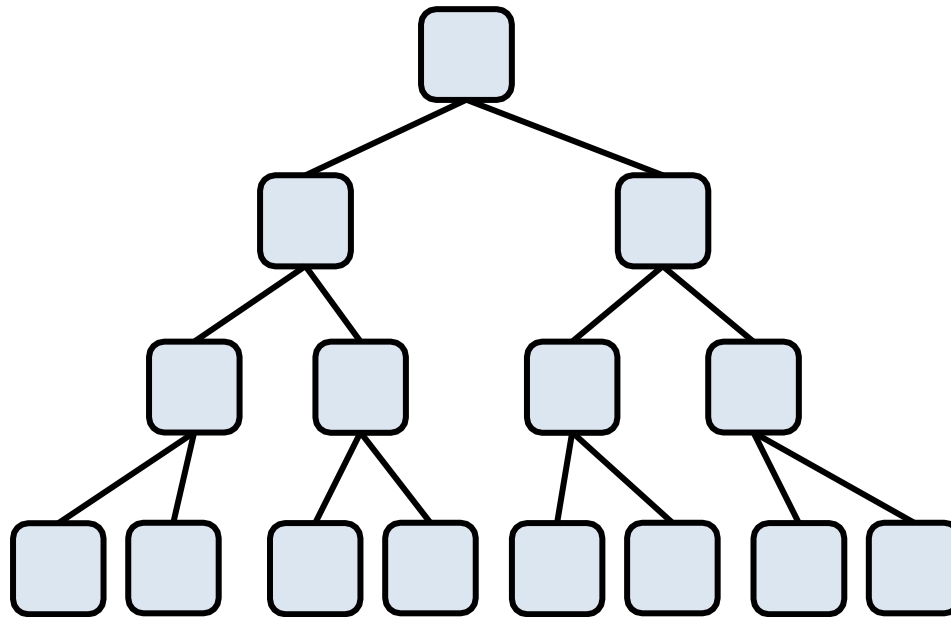
Full Binary Tree



Complete Binary Tree

Types of Binary Trees

- 포화 이진 트리 (Perfect Binary Tree)
 - 모든 리프 노드의 레벨이 서로 같음
 - 모든 내부 노드의 자식 노드 수가 2임

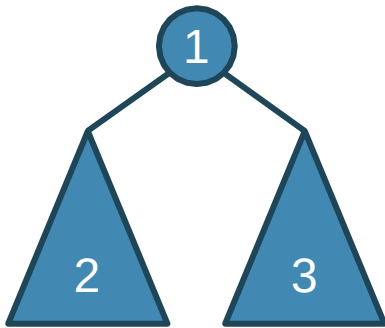


Perfect Binary Tree

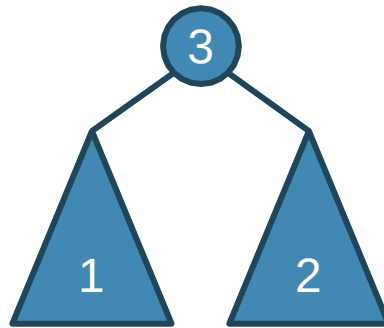
Tree Traversal Algorithms

Tree Traversal (트리 순회)

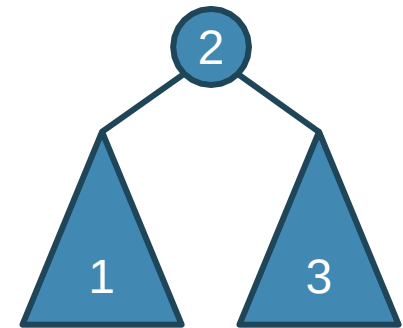
- 트리 순회
 - 트리의 모든 노드를 일정한 규칙에 따라 정확히 한번씩 방문하는 과정
- 순회 종류
 - 전위 순회 (Pre-order): 노드를 방문하고, 자식들을 순회
 - 후위 순회 (Post-order): 자식들을 방문하고, 노드를 방문
 - 중위 순회 (In-order): 왼쪽 서브트리 방문 후, 노드 방문, 그리고 나서 오른쪽 서브트리 방문



Pre-order traversal



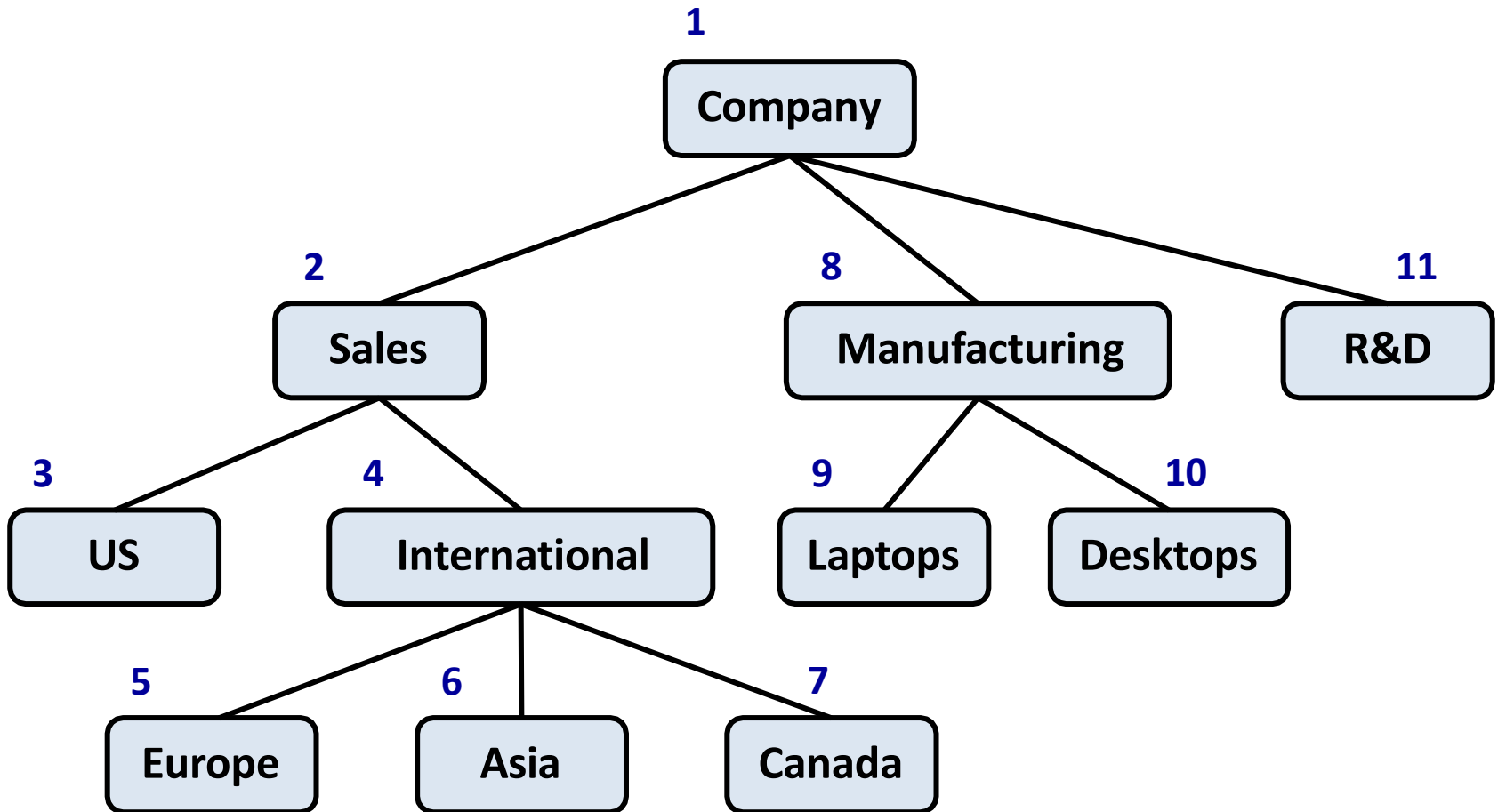
Post-order traversal



In-order traversal

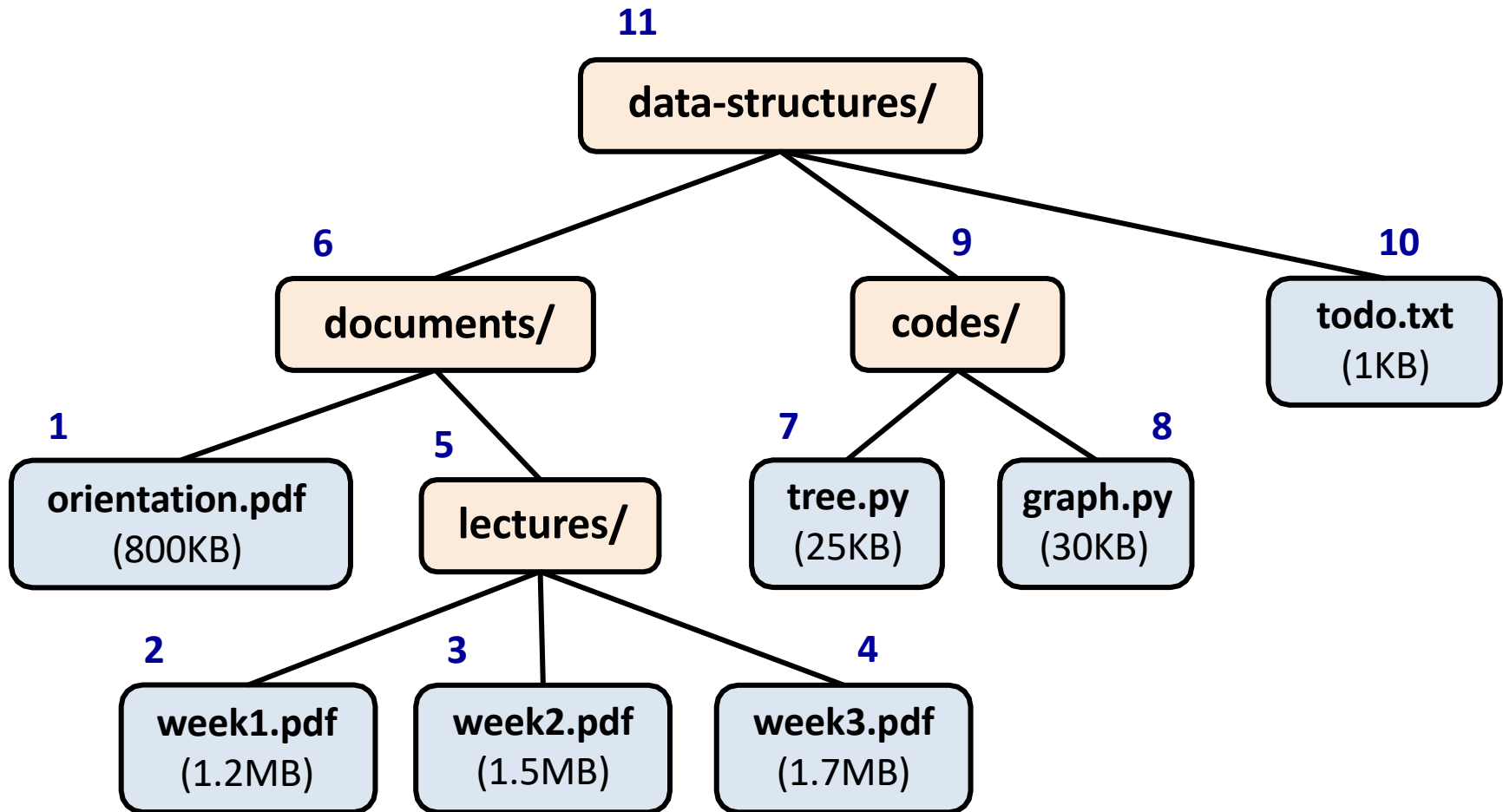
Pre-order Traversal (전위 순회)

- 특정 노드를 **자손 노드들보다 먼저 방문**
- 활용 예시: 계층적 문서 구조 출력, prefix 표기식 등



Post-order Traversal (후위 순회)

- 특정 노드를 **자손 노드들을 모두 방문한 뒤에 방문**
- 활용 예시: 디렉토리에서 사용 중인 전체 공간 계산, postfix 표기식 등



Pre-order vs. Post-order

Pre-order traversal

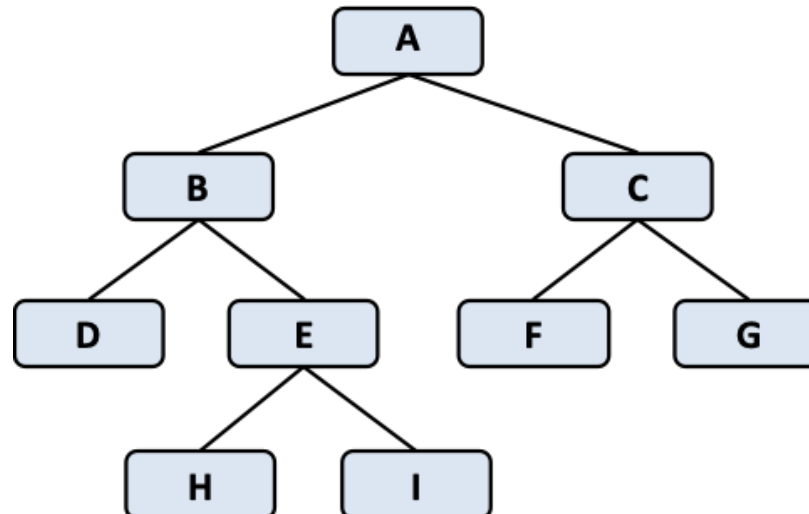
Algorithm binaryPreorder(v)

```
1: visit(v)
2: if hasLeft(v)
3:   binaryPreorder(left(v))
4: if hasRight(v)
5:   binaryPreorder(right(v))
```

Post-order traversal

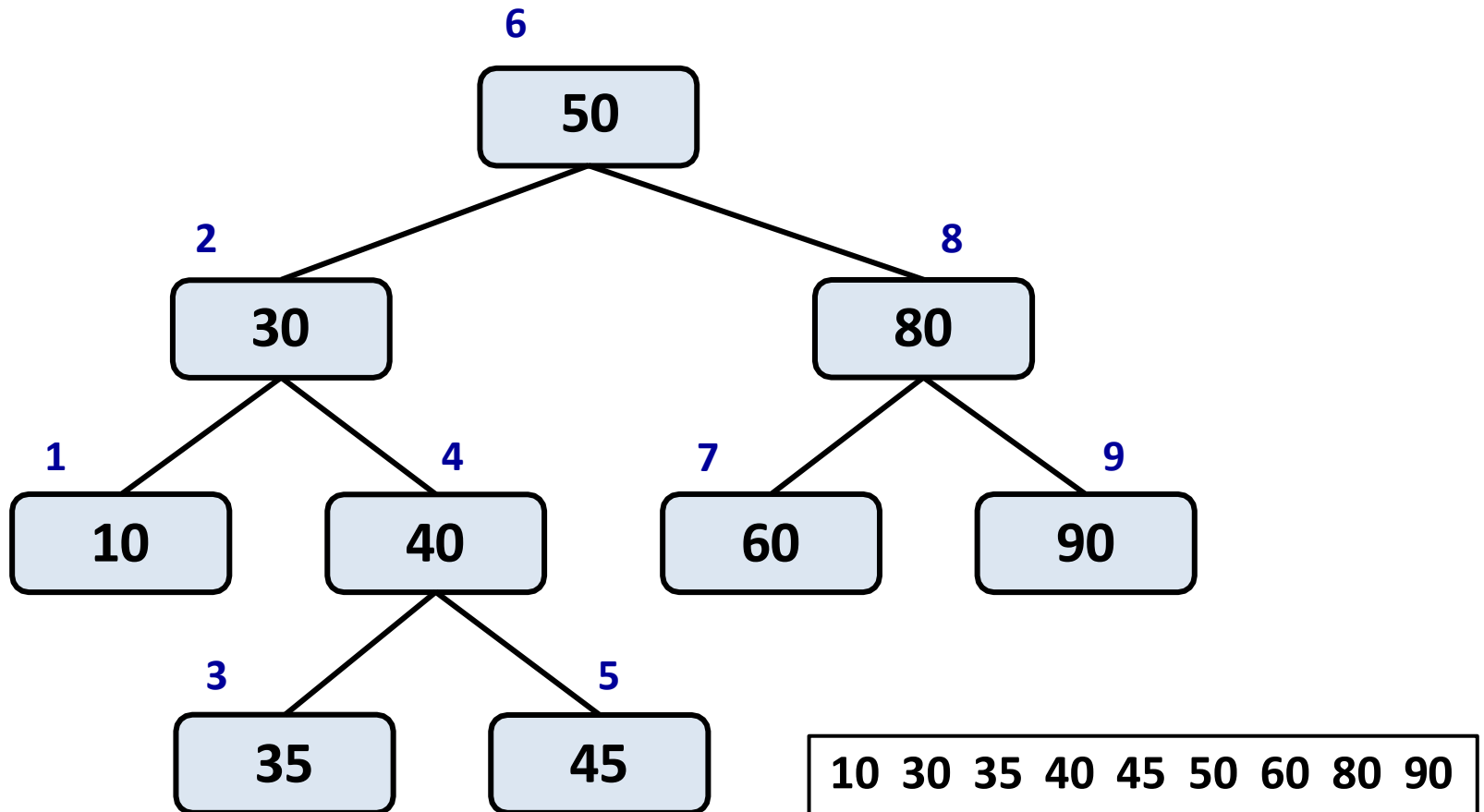
Algorithm binaryPostorder(v)

```
1: if hasLeft(v)
2:   binaryPostorder(left(v))
3: if hasRight(v)
4:   binaryPostorder(right(v))
5: visit(v)
```



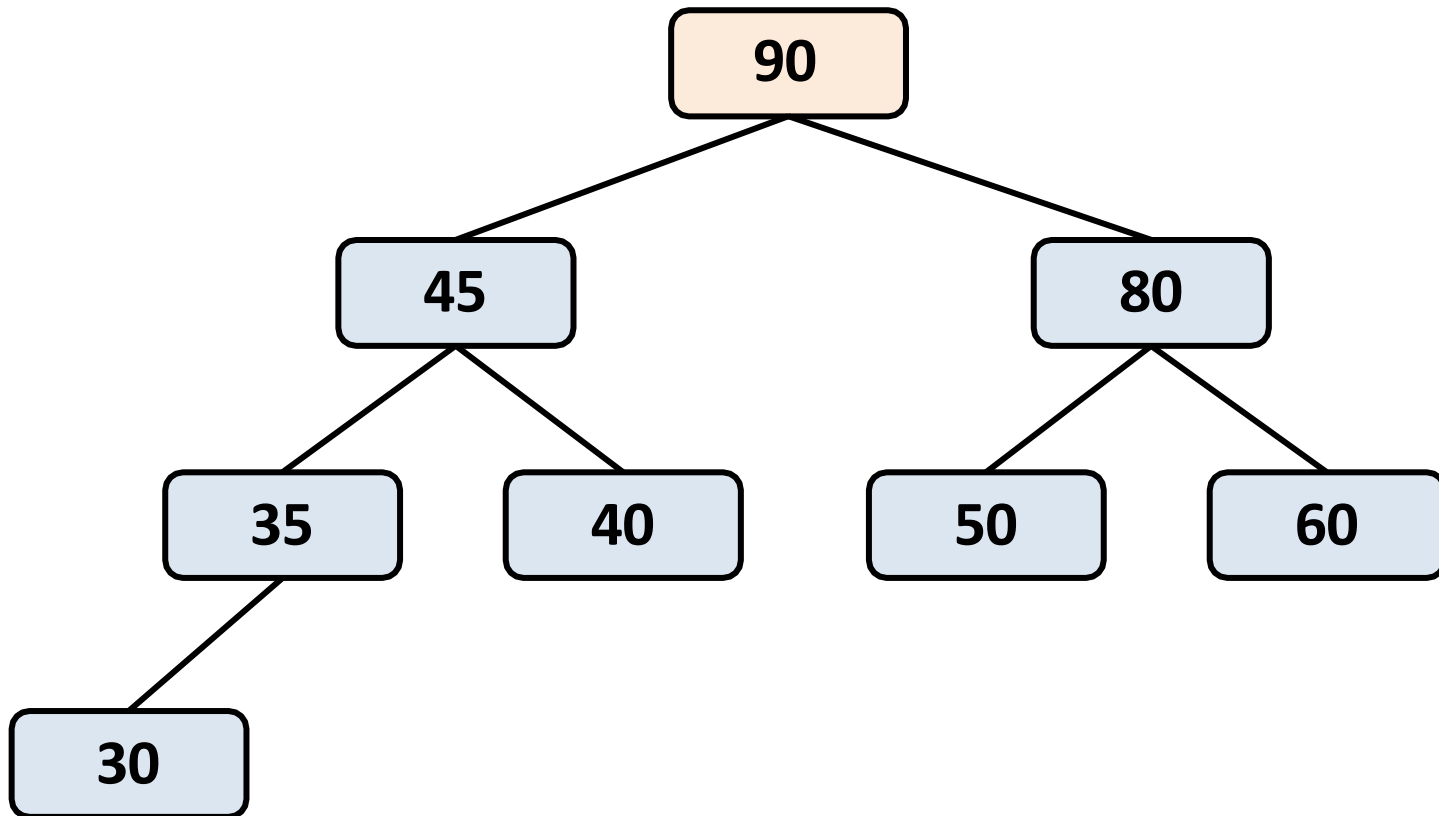
In-order Traversal (중위 순회)

- 한 노드를 왼쪽 서브트리를 모두 방문한 뒤, 그 다음에 방문하고, 이후 오른쪽 서브트리 방문
- 활용 예시: 이진 탐색 트리(Binary Search Tree)에서 키 값을 정렬된 순서로 얻을 수 있음



Preview: Heap & Priority Queue (힙 / 우선순위 큐)

- 완전 이진 트리 (Complete Binary Tree) 구조에 기반
- 최대값 또는 최소값을 효율적으로 찾을 수 있도록 설계된 자료 구조



Max Heap

Look ahead

Binary Search Tree